

Beam Search Scheduling for DAG Computation Graphs

Tile-simulated scheduling under limited fast memory

Wenqi Deng

Peking University

Google MLSys Competition 2026

Track A: Systems Engineering

Team and Acknowledgement

- **Presenter: Wenqi Deng**
- School of EECS, Peking University
- Undergraduate student, Turing Class



Advisor: Prof. Xupeng Miao
Peking University

Acknowledgement

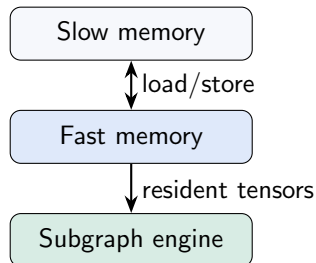
I would like to thank Prof. Xupeng Miao for his valuable guidance and support.

Problem Definition

Objective

Minimize total latency while every execution step fits in fast memory.

- Slow memory: large but bandwidth-limited.
- Fast memory: fast but capacity-limited
- Fusing ops creates ephemeral internal tensors.
- Each subgraph is assigned a granularity $[w, h, k]$.

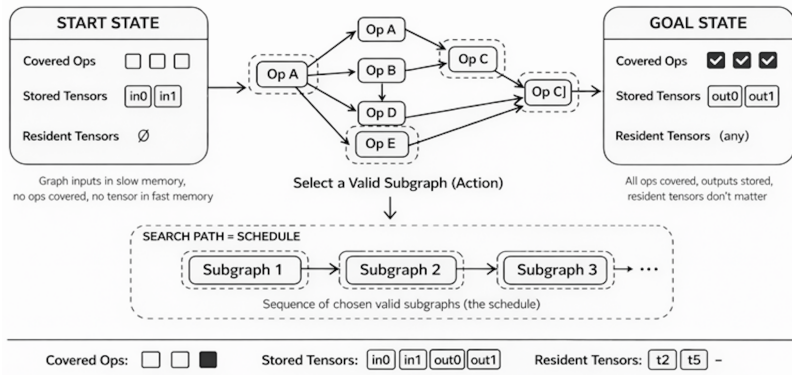


Search Model

Search state representation

State(covered ops, stored tensors, resident tensors)

- Start: graph inputs are stored in slow memory.
- Transition: choose one feasible subgraph.
- Goal: all ops covered and outputs stored.



Beam Search Loop

- 1 Expand each beam state with generated candidates.
- 2 Evaluate each candidate through the simulator.
- 3 Append the chosen subgraph to form a next state.
- 4 Deduplicate equivalent states.
- 5 Keep the best states by estimated total latency.

Priority function

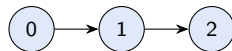
$$\text{priority}(\text{State}) = \text{latency}(\text{State}) + 0.1 \times \sum_{\text{uncovered ops}} \text{base_cost}$$

Candidate Generation

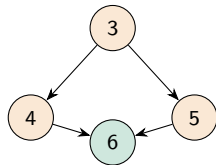
- Ready single operations.
- Linear pointwise chains.
- Pointwise backward cones.
- Shallow recomputation cones.

Design principle

Generate a small set of high-value subgraph templates, then let the evaluator reject illegal or expensive ones.

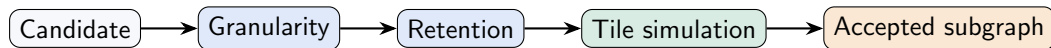


chain



backward cone

Candidate Evaluation



- Enumerate w, h choices, and choose the largest available k .
- Enumerate retain sets on boundary outputs.
- Reject any combination with an OOM step.
- Return the lowest-latency feasible subgraph.

Tile Simulator

- Trace required tensor rectangles from the subgraph tip tensor.
- Track external inputs, boundary outputs, retained tensors and accumulators.
- Use rectangle union area per tensor to avoid double counting.
- Validate every spatial tile and every split-k step.

Capacity rule for OOM correctness

$$|R_{\text{resident}} \cup R_{\text{retain}} \cup R_{\text{input}} \cup R_{\text{output}} \cup R_{\text{accumulator}}| \leq C_{\text{fast}}$$

Retention is inter-subgraph persistence

Only selected boundary outputs stay in fast memory for the next scheduling step.

- Retain candidates are boundary outputs with future consumers.
- Non-retained boundary outputs are stored back to slow memory.
- Resident tensors must be consumed at the immediate following tensor.

Validation Results

Benchmark	Latency	Peak fast memory
mlsys-2026-1	471,501	49,152
mlsys-2026-5	961,195	24,576
mlsys-2026-9	167,530,988	49,152
mlsys-2026-13	166,408,847	65,536
mlsys-2026-17	46,386,493	311,296

Verification

Solutions were validated on the public benchmarks with our verifier, and the reported latencies are computed by our cost model.

Takeaways

- Beam search avoids one-step greedy decisions.
- Candidate templates keep the search space manageable.
- Tile simulation gives robust OOM rejection.
- Retention reduces slow-memory traffic while preserving physical state semantics.

Potential improvements

Richer MatMul-chain candidates, memory-aware heuristics, and stronger retention scoring.